

Chương 1: PHÂN TÍCH ĐỘ PHỨC TẠP THUẬT TOÁN

Bài toán 1.1.1. Bài toán tìm dãy con lớn nhất

Cho dãy số $a_1, a_2, \dots, a_n$, dãy số $a_i, a_{i+1}, a_{i+2}, \dots, a_j$ với $1 \leq i \leq j \leq n$ là một dãy con của dãy số đã cho, tổng $\sum_{k=i}^j a_k$ được gọi là trọng lượng của dãy. Bài toán đặt ra tìm dãy con có trọng lượng lớn nhất.

Các cách tiếp cận giải quyết bài toán

1.1.1. Thuật toán trực tiếp

```
pair<int, vector<int>> BruteForce1(vector<int> a)
{
    int n = a.size();
    int maxSum = INT_MIN, head = -1, tail = -1;
    for (int i = 0; i < n; ++i)
    {
        for (int j = i; j < n; ++j)
        {
            int tmpSum = 0;
            for (int k = i; k <= j; ++k){
                tmpSum += a[k];
                if (tmpSum > maxSum)
                {
                    maxSum = tmpSum;
                    head = i;
                    tail = j;
                }
            }
        }
    }
    vector<int> res(a.begin()+head, a.begin()+tail+1);
    return {maxSum, res};
}
```

1.1.2. Cải tiến thuật toán trực tiếp

```
pair<int, vector<int>> BruteForce2(vector<int> a)
{
    int n = a.size();
    int maxSum = INT_MIN, head = -1, tail = -1;
    for (int i = 0; i < n; ++i)
    {
        int tmpSum = 0;
        for (int j = i; j < n; ++j)
        {
```

```

        tmpSum += a[j];
        if (tmpSum > maxSum)
        {
            maxSum = tmpSum;
            head = i;
            tail = j;
        }
    }
    vector<int> res(a.begin()+head, a.begin()+tail+1);
    return {maxSum, res};
}

```

1.1.3. Chia để trị dùng đệ quy

```

pair<int,int> MaxLeft(vector<int> a, int left, int mid)
{
    int maxSum = INT_MIN, sum = 0, left_index=-1;
    for (int i = mid; i >= left; --i){
        sum += a[i];
        if (sum > maxSum)
        {
            maxSum = sum;
            left_index = i;
        }
    }
    return {maxSum, left_index};
}

/// mid in this function is mid+1 in real case.
pair<int,int> MaxRight(vector<int> a, int mid, int right) {
    int maxSum = INT_MIN, sum = 0, right_index=-1;
    for (int i = mid; i <= right; ++i){
        sum += a[i];
        if (sum > maxSum)
        {
            maxSum = sum;
            right_index = i;
        }
    }
    return {maxSum, right_index};
}

pair<int, vector<int>> MaxSubSequence(vector<int> a, int left, int
right)
{
    if (left == right) return {a[left], {a[left]}};
    int mid = left + (right-left)/2;
    pair<int, vector<int>> wL = MaxSubSequence(a, left, mid);

```

```

pair<int, vector<int>> wR = MaxSubSequence(a, mid+1, right);
pair<int, int> mL = MaxLeft(a, left, mid);
pair<int, int> mR = MaxRight(a, mid+1, right);
if (wL.first > wR.first && wL.first > mL.first+mR.first)
    return wL;
else if (wL.first < wR.first && wR.first > mL.first+mR.first)
    return wR;
else{
    vector<int> res(a.begin()+mL.second, a.begin()+mR.second+1);
    return {mL.first+mR.first, res};
}
}

```

1.1.4. Quy hoạch động

```

pair<int, vector<int>> dynamicProgramming1(vector<int> a)
{
    int n = a.size();
    vector<int> s(n);
    s[0] = a[0];
    for (int i = 1; i < n; ++i)
        s[i] = max(s[i-1]+a[i], a[i]);
    auto maxsum = max_element(s.begin(), s.end());

    /// trace
    int j = -1;
    for (int i = 0; i < n; ++i)
        if (s[i] == *maxsum)
            j = i;

    vector<int> subseq;
    for (int i = j; i > 0; --i)
    {
        subseq.push_back(a[i]);
        if (a[i] > s[i-1]+a[i])
            break;
    }
    if (a[1]<a[0]+a[1])
        subseq.push_back(a[0]);
    return {*maxsum, subseq};
}

```